

Python Modules & Packages

Complete Beginner-to-Builder Master Notes & Architecture Guide

Track: Python from Scratch •
03_Modules&Packages

Review Target: 30
Minutes

Scope: ClassWork, HomeWork (01-04), Assignment
(HMS)

The 30-Minute Fast-Track Revision Plan

00:00 – 05:00 min	Module Fundamentals: What is a module, import syntax flavors, aliasing, and function collision rules.
05:00 – 10:00 min	Standard Modules: System manipulation with <code>os</code> , execution timing with <code>time</code> , calendars with <code>calendar</code> .
10:00 – 18:00 min	Package Architecture: <code>__init__.py</code> role, relative imports (<code>.</code>), and dynamic path resolution with <code>sys.path</code> .
18:00 – 25:00 min	Project Workflows: Real-world implementations (Employee, Student, Number, Shopping, Hospital Management).
25:00 – 30:00 min	Troubleshooting & Interview Q&A: Bytecode caches (<code>__pycache__</code>), namespace collision fixes, best practices.

1. Python Modules: Concept & Syntax

A **Module** in Python is simply a single file ending with `.py` containing functions, variables, or classes designed to be reused in other programs.

Why Use Modules?

- **Code Reusability:** Write a utility once (e.g. math or validation functions), import it everywhere.
- **Maintainability & Cleanliness:** Keeps `main.py` lightweight, readable, and free of cluttered utility logic.
- **Namespace Separation:** Prevents naming conflicts between different parts of a system.

4 Ways to Import Modules (With Code Examples)

Import Style	Syntax Example	Behavior & When to Use
Direct Import	<pre>import module1 module1.add(2, 3)</pre>	Brings the entire module into memory. Must prefix calls with module name. Cleanest namespace safety.
Import with Alias	<pre>import module1 as m1 m1.add(2, 3)</pre>	Shortens long module names (e.g., <code>import numpy as np</code>). Easier to type repeatedly.
Selective Import	<pre>from module1 import add, func add(2, 3)</pre>	Imports only specific functions directly into current namespace. No need to type module prefix.
Wildcard Import	<pre>from module1 import * add(2, 3)</pre>	Imports all public names. <i>Not recommended</i> in production because it causes invisible namespace collisions.

The Critical Name Collision Rule ("Last Imported Wins")

In our classwork (`module2.py`), we imported two functions having the exact same name `func()` :

```
from module1 import func, add # module1 has def func() -> "Function is Module1"
from module3 import func      # module3 has def func() -> "Function in Module3"

print(func()) # OUTPUT: "Function in Module3" (module3 will win!)
```

Key Rule: When two imported modules define a function or variable with identical names, Python binds the name to whichever import executed **last**. To prevent this, either import the module directly (`module1.func()`) vs `module3.func()`) or use aliases (`from module1 import func as f1`).

2. Built-in Standard Library Modules Explored

A. The `os` Module (Operating System Navigation)

The `os` module provides direct interaction with the host operating system's filesystem.

```
import os

cwd = os.getcwd()          # Returns current working directory string
os.mkdir("Hiii")           # Creates a new directory named 'Hiii'
files = os.listdir()       # Returns list of files and folders in directory
os.rmdir("Hiii")           # Deletes directory (must be empty!)
os.chdir("../")            # Navigates up one folder level
```

B. The `time` Module (Execution Time & Scheduling)

Used to measure performance, pause execution, and inspect system clocks.

```
import time

# 1. Measure Code Performance
start = time.time()          # Seconds since Unix Epoch (Jan 1, 1970)
for i in range(100000):
    pass
end = time.time()
print("Loop duration:", end - start, "seconds")

# 2. Pausing Execution
time.sleep(3)                # Halts thread execution for exactly 3 seconds

# 3. Readable Timestamp Formats
print(time.ctime())          # Current human string: 'Wed Sep 20 13:12:00 2026'
print(time.ctime(1782678388)) # Converts any specific epoch timestamp to human string

# 4. Time Structures
x = time.localtime()         # Returns struct_time with tm_year, tm_mon, tm_mday, etc.
print("Current Day:", x.tm_mday)
print(time.gmtime())         # Returns UTC time struct
```

C. The `calendar` Module (Dates, Leap Years & Calendars)

Generates text-based calendars and computes date ranges.

```
import calendar

print(calendar.calendar(2026))          # Prints entire year calendar
print(calendar.month(2026, 7))          # Prints single month (July 2026)
print(calendar.isleap(2024))            # Returns True / False
weekday, days = calendar.monthrange(2026, 7) # Returns (start_weekday, total_days_in_month)
calendar.setfirstweekday(calendar.SUNDAY) # Configures starting column
```

3. Python Packages & Architecture

A **Package** is a directory containing one or more Python modules plus a special file named `__init__.py`.

Why do we need `__init__.py` ?

- It formally marks the directory as a Python package that can be imported.
- It executes automatically upon package import, making it ideal for package setup or initialization.
- It defines the package's public API by re-exporting internal module functions so users don't need to know the inner folder structure.

3 Architectural Patterns Explored in ClassWork

Pattern	Structure & Setup	How <code>main.py</code> Uses It
Pattern 1: Auto-Exposing API (Folder <code>02</code>)	<code>mypackages/__init__.py</code> has: <pre>from .module1 import func1 from .module2 import func2</pre>	<pre>import mypackages mypackages.func1() mypackages.func2()</pre> <i>Cleanest for end-users!</i>
Pattern 2: Explicit Submodule (Folder <code>03_Main_Same_Folder</code>)	<code>mypackages/__init__.py</code> is empty. Modules stay separate.	<pre>from mypackages import module1, module2 module1.func1() module2.func2()</pre>
Pattern 3: Anywhere Import via <code>sys.path</code> (Folder <code>04_Main_Anywhere</code>)	<code>main.py</code> is located in a completely different directory or folder.	<pre>import sys sys.path.append("/path/to/folder") from mypackages import module1</pre> <i>Allows OS-wide imports!</i>

Understanding `sys.path` : Python only looks for modules in the current directory and directories listed in `sys.path` . When our script lives in a child directory (like `04_Main_Anywhere/main/`) but the package is in a sibling or distant directory, Python throws `ModuleNotFoundError` . Appending the package's parent folder to `sys.path` informs Python's import engine where to look.

4. Hands-on Modular Projects (HomeWork Breakdown)

HomeWork 01 Employee Data Processing System

Folder: `HomeWork_26Jun/01/` • **Package:** `Package/`

Modules Built:

- `add.py` : Takes multiple employee ages and names, populates module-level lists using `global`.
- `second_high.py` : Finds the second highest age in $O(n)$ time using two pointers (`highest` , `second = float('-inf')`) without sorting.
- `seniors.py` : Filters and counts employees older than 60 years.
- `remove_duplicates.py` : Deduplicates ages preserving uniqueness.
- `starting_name.py` : Counts employee names starting with vowels (A, E, I, O, U).
- `longest_empname.py` : Traverses names list to find the longest employee name string.

Key Concept: Using Python's modern `match choice: case 1: ...` menu-driven structure in `main.py`.

HomeWork 02 Student Result Management System

Folder: `HomeWork_26Jun/02/` • **Package:** `Packages/`

Modules Built:

- `for_add.py` : Uses variable positional arguments `def add_details(name, roll, *marks)` and packs them into a student dictionary.
- `for_total.py` : Sums up subject marks.
- `for_per.py` : Calculates student percentage against total maximum possible marks.
- `for_grade.py` : Maps percentage to letter grades (A+, A, B, Fail).
- `for_max.py` & `for_min.py` : Computes highest and lowest scoring subject marks.
- `for_result.py` : Prints the formatted final report card.

Key Concept: Argument unpacking and passing `*marks` dynamically across module functions.

HomeWork 03 Number Analysis System

Folder: `HomeWork_26Jun/03/` • **Package:** `Packages/`

Modules Built:

- `for_perfect.py` : Checks if sum of proper positive divisors equals the number (e.g. $6 = 1 + 2 + 3$).
- `for_prime.py` : Validates prime status through trial division.
- `for_reverse.py` : Reverses integer digits mathematically or via string slicing.
- `for_factorial.py` : Calculates $n!$.
- `for_factors.py` : Generates list of all divisors of a number. Unpacked in `main.py` with `*(factors(n))`.

HomeWork 04 Online Shopping System**Folder:** HomeWork_26Jun/04/ • **Package:** Packages/**Modules Built:**

- `for_add_customer.py` : Records customer name, email, and mobile contact.
- `for_add_product.py` : Uses keyword arguments (`name=name`, `cat=cat`, `price=price`).
- `for_invoice.py` : Calculates billing with 10% tax surcharge.
- `for_total.py` : Computes order subtotal using `def total(*prices)` .
- `for_profile.py` : Renders comprehensive customer profile with keyword arguments.

5. Capstone Assignment: Hospital Management System (HMS)

The culmination of the module is an enterprise multi-package architecture located in `Assignments/01/`.

Directory Architecture

```
Assignments/01/
├── main.py                # Orchestrates application menu and user workflow
├── patient/
│   ├── __init__.py        # Re-exports: add_patient, display_patients, search_patient
│   └── patient_module.py  # In-memory dictionary database for patient records
├── doctor/
│   ├── __init__.py        # Re-exports: add_doctor, display_doctors
│   └── doctor_module.py   # Doctor registration with specialization & fees
├── appointment/
│   ├── __init__.py        # Re-exports: book_appointment, show_appointments
│   └── appointment_module.py # Relational binding between Patient ID and Doctor ID
└── billing/
    ├── __init__.py        # Re-exports: generate_bill
    └── billing_module.py   # Calculates bill with try-except validation
```

Key Architectural Lessons from HMS

1. **Clean Re-Exports in `__init__.py`** : Instead of requiring `main.py` to do deep imports like: `from patient.patient_module import add_patient`, each package's `__init__.py` contains:

```
from .patient_module import add_patient, display_patients, search_patient
```

This allows `main.py` to stay clean and decoupled:

```
from patient import add_patient, display_patients, search_patient
```

2. **Defensive Programming with `try...except` in `billing_module.py`** :

```
try:
    c_Charges = float(input("Enter Consultation Charges: "))
    m_Cost = float(input("Enter Medicine Cost: "))
    t_Charges = float(input("Enter Test Charges: "))
except ValueError:
    print("Invalid input for charges. Please enter numeric values.")
    return
```

Prevents the program from crashing if a user enters non-numeric text for bills.

6. Quick Revision FAQs & Interview Takeaways

Question	Precise Answer for Exams & Interviews
What is the difference between a Module and a Package?	A Module is a single <code>.py</code> file containing executable code. A Package is a directory containing multiple modules and an <code>__init__.py</code> file that coordinates them.
What is the purpose of <code>__pycache__</code> ?	Python compiles imported <code>.py</code> files into bytecode (<code>.pyc</code> files) stored inside <code>__pycache__</code> . This speeds up future import operations because compilation can be skipped if the source hasn't changed.
What happens if two modules have a function with the same name?	If imported with <code>from mod import func</code> , the last imported one overwrites previous bindings. Use <code>import mod1; import mod2</code> or aliasing (<code>as</code>) to prevent collisions.
Why use <code>if __name__ == '__main__':</code> ?	It ensures that certain code (like test runs or demos) executes only when the script is run directly from the terminal, but remains silent when the file is imported by another module.
How does Python know where to find packages?	It inspects directories in <code>sys.path</code> : the current script directory, <code>PYTHONPATH</code> environment variables, and installed <code>site-packages</code> . You can dynamically append to it with <code>sys.path.append()</code> .